

# **SELF-ASSESSMENT & WEEK 13–14 RESEARCH SPRINT PROPOSAL**

**Name:** Jadhav Guneet Kaur

**Project:** HRAF Middleware & Real-Time HARM Safety Shield

**Focus:** Self-Assessment & Research Sprint Planning

## **SECTION 1: ONE-PAGE SELF-ASSESSMENT**

### **1. Core Technical Strengths (What Was Built)**

- **Deterministic Middleware & Multi-Stage Safety Architecture:** Designed and implemented the core 3-stage HARM safety shield engine. This includes Stage 1 structural Pydantic schema validation with single re-prompt budget ( $N = 1$ ), Stage 2 spatial workspace 3D target projection clipping ( $R_{\min} = 0.2\text{m}$ ,  $R_{\max} = 1.0\text{m}$ ), and Stage 3 kinematic velocity scaling ( $\leq 0.8 \text{ rad/s}$ ) with safe zero-velocity hold fallbacks.
- **Asynchronous Queue Management & Telemetry Profiling:** Engineered async thread bridges decoupling low-frequency VLM inference (2.0 Hz) from high-frequency Inverse Kinematics execution loops (50 Hz). Successfully verified execution stability through telemetry profiling, maintaining a flat 85 MB RAM footprint, bounded zero-leak queue rotation, and zero system exceptions across 295 continuous frames.
- **VLM Interception Testing & Latency Optimization:** Built real-world test harnesses evaluating 50 VLM calls across E1, E2, and E3 environment scenarios. Achieved a local deterministic intercept latency of  $< 5 \text{ ms}$  (Average:  $\sim 1.2 \text{ ms}$ ), providing a  $> 300 \times$  speedup factor in safety resolution compared to cloud VLM re-queries (1500 ms – 3000 ms).
- **Closed-Loop Kinematic Verification:** Developed real-time monitoring routines that dynamically detect joint velocity spikes and automatically calculate trajectory time scaling to smooth motor acceleration before issuing hardware torque commands.

### **2. Engineering Growth Areas / Weak Points**

- **Native ROS 2 Package Build Architectures:** Relied on PYTHONPATH exports for standalone Python middleware execution rather than creating a fully standard colcon-built ROS 2 package containing a native package.xml, CMake/Python build configurations, and automated setup.py entry points.
- **Executor Loop Synchronization & Threading:** Experienced challenges managing rclpy executor loops embedded inside asynchronous asyncio routines, requiring careful thread isolation to avoid blocking ROS 2 callback execution under heavy frame rates.

- **Simplistic Spatial Obstacle Models:** Stage 2 spatial safety currently relies on a dual-sphere radial shell approximation ( $R_{\min} = 0.2\text{m}$ ,  $R_{\max} = 1.0\text{m}$ ) rather than dynamic URDF link geometry or 3D volumetric obstacle representations.

### 3. Technical Things Still Unclear

- **Zero-Copy Memory Buffers in ROS 2:** The exact architectural mechanisms for implementing zero-copy shared memory transport when passing high-bandwidth RGB camera feeds (/camera/image\_raw) across Python and C++ ROS 2 nodes without inducing heap allocation spikes.
- **Token-Level Constrained Generation vs. Re-prompting:** The exact performance and latency trade-offs of using direct logit-level schema constraints (e.g., Outlines / Guidance) vs. runtime  $N = 1$  re-prompting loops during tight  $< 20$  ms safety-critical control windows.
- **Dynamic URDF Trajectory Verification:** How to efficiently validate joint limit matrices and self-collision lookup tables in real time (50 Hz) without incurring the computational overhead of offloading to full physics simulators like Gazebo or Isaac Sim.
- **Callback Queue Priority & Latency Jitter:** How rclpy's MultiThreadedExecutor prioritizes callback group execution queues when high-rate camera frame subscriptions and telemetry publishers contend for CPU cycles on resource-constrained embedded targets.

## SECTION 2: WEEK 13–14 LEARNING PLAN & RESEARCH SPRINT PROPOSAL

### 1. Specific Learning Plan

#### Week 13: Advanced ROS 2 Middleware Packaging & Multi-Threaded Execution

- **Concrete Plan:** Complete *The Construct ROS 2 Advanced Python Development* course modules focusing on colcon build system architectures, custom .msg/.srv interface creation, and MultiThreadedExecutor implementation to eliminate event-loop blocking.
- **Target Outcome:** Refactor hraf-middleware from a standalone script repository into a clean, colcon-buildable ROS 2 package with proper package.xml declarations, setup.py entry points, and ROS 2 launch files (.launch.py).

#### Week 14: Real-Time Collision Geometry & Guided Schema Integration

- **Concrete Plan:** Complete *The Construct ROS 2 Motion Planning & TF2* modules to master dynamic transform lookups, URDF link geometry parsing, and zero-copy node integration.

- **Target Outcome:** Upgrade HARM Stage 2 to support dynamic URDF link limit checks and prototype logit-level schema guidance for incoming VLM JSON strings.

## 2. Independent HRAF Task Ownership

During the Week 13–14 Research Sprint, I can independently own and deliver the following specific HRAF engineering tasks:

- **Native ROS 2 Middleware Package Refactoring:** Standardize the repository into a fully compliant ROS 2 package (package.xml, setup.py, and .launch.py scripts) that seamlessly builds and integrates with e2\_environment via standard colcon build.
- **HARM Stage 2 URDF Bounds Upgrade:** Replace the current spherical shell approximation with a link-aware kinematic limit checker that parses URDF joint boundaries.
- **Logit-Guided JSON Enforcement Engine:** Prototype a local guided-decoding filter (using Outlines/Guidance) to eliminate reliance on  $N = 1$  network re-prompting loops for malformed VLM JSON schemas.
- **Automated CI Integration & Benchmarking Suite:** Author automated ROS 2 test launch scripts and pytest harnesses to allow other team members to run stress tests, memory profiling, and catch-rate benchmarks with a single command.